

CIVENTRAL Urban Planning

Module Build Reference

How every module is actually built — the technique/library behind it, not just a file list. Two build patterns exist (see `CLAUDE.md`): **local-DB modules** (own MySQL table, built in PHP with the repository/service/API layers) and **production-proxied modules** (no local table — the request is forwarded to `civentral.tech`, which owns `auth/RBAC/users/roles/audit`).

Resident Management

The master identity registry. Plain CRUD over two tables (residents, households) via `ResidentRepository/HouseholdRepository` and PDO prepared statements — no ORM. Search fields are combined with `CONCAT_WS(' ', ...)` `LIKE :search` in one placeholder rather than repeating `:search`, because PDO runs with `EMULATE_PREPARES` disabled and rejects a reused named placeholder. Deleting a record never removes the row — `DELETE` flips a status column to archived (soft-delete), consistent across every local module. Resident documents (IDs, proof-of-address) go through `FileUploadService`: extension/MIME whitelist, 5MB cap, randomly generated stored filename so the original filename is never trusted.

Built in: `src/Repositories/ResidentRepository.php`, `HouseholdRepository.php`, `ResidentDocumentRepository.php`; `src/Services/ResidentService.php`, `HouseholdService.php`; `api/employee/residents.php`, `households.php`, `resident-documents.php`; `pages/resident/`; `assets/js/resident/`, `assets/js/household/`.

Housing Management

Tracks a housing unit through its whole lifecycle. The interesting part isn't CRUD, it's the business logic layered on top:

- **Eligibility scoring** — `BeneficiaryService` computes a 0–100 weighted score from income, family size, vulnerability category, and household tenure type on every save, rather than a reviewer eyeballing an application.
- **Duplicate-detection** — a query blocks a second open application from the same resident or household before it's inserted.
- **Automatic cross-module updates** — awarding a unit to a beneficiary flips that unit's `occupancy_status` to `Occupied` automatically. A relocation record does the same trick in reverse: `HousingRelocationService` is constructed with `HousingOccupancyService` injected into it, so recording a relocation closes the old occupancy row and opens a new one in the same call — no separate manual step in `Occupancy`.

Built in: `HousingUnitRepository.php`, `HousingBeneficiaryRepository.php`, `HousingOccupancyRepository.php`, `HousingRelocationRepository.php`, `BeneficiaryDocumentRepository.php`; `HousingService.php`, `BeneficiaryService.php`, `HousingOccupancyService.php`, `HousingRelocationService.php`; `api/employee/housing-units.php`, `housing-beneficiaries.php`, `beneficiary-documents.php`, `housing-occupancy.php`, `housing-relocations.php`; `pages/housing/`; `assets/js/housing/`, `housing-beneficiaries/`, `housing-occupancy/`, `housing-relocations/`.

Field Survey

Built around a **polymorphic subject reference** instead of a separate table per inspection target: an assignment stores `subject_type` (Resident / Household / Site) and `subject_id`, so one schema covers inspecting a person, a household, or a free-text site address without three near-duplicate tables. Recording a result auto-flips its assignment to Completed — no separate "mark as done" action. Survey History isn't a table at all — `FieldSurveyAssignmentRepository::getHistoryForSubject()` merges assignments and results into one timeline at query time.

Built in: `FieldSurveyFormRepository.php`, `FieldSurveyAssignmentRepository.php`, `FieldSurveyResultRepository.php`, `FieldSurveyPhotoRepository.php`; `FieldSurveyFormService.php`, `FieldSurveyAssignmentService.php`, `FieldSurveyResultService.php`; `api/employee/field-survey-{forms,assignments,results,photos}.php`; `pages/field-survey/`; `assets/js/field-survey-{forms,assignments,results,history}/`.

Urban Planning

The most rule-heavy module. Zoning Clearance runs an automated **conformity check**:

`ZoningConformityService` reads a seeded regulation matrix (`ZoningUseRegulationRepository` — which land uses are Permitted/Conditional/Prohibited per zone, plus numeric limits on height/setback/floor-area-ratio/lot-occupancy) and evaluates the application's figures against it, both live as the applicant fills the form and again permanently on save, citing the specific violated limit. A fee is computed from a formula (base amount + a per-square-meter rate that varies by use category). Approval generates a verification code and unlocks a printable Certificate of Zoning Compliance (server-rendered HTML, no PDF library); denial unlocks a Notice of Non-Conformity citing the exact findings. Permit review is a multi-stage, discipline-based workflow (structural, electrical, sanitary, etc. each reviewed separately) where every stage change is written to an append-only log table with reviewer, role, and required remarks — a real audit trail rather than a status flag that gets overwritten.

Built in: `DevelopmentPlanRepository.php`, `ZoningClearanceRepository.php`, `ZoningUseRegulationRepository.php`, `UrbanProjectRepository.php`, `InfrastructureRecordRepository.php`, `PermitApplicationRepository.php`, `PermitPlanDocumentRepository.php`; matching `*Service.php` classes plus `ZoningConformityService.php`; `api/employee/development-plans.php`, `zoning-clearances.php`, `urban-projects.php`, `infrastructure-records.php`, `permit-applications.php`, `permit-plan-documents.php`, `planning-documents.php`; `pages/urban-planning/`; `assets/js/urban-planning/`, `zoning-clearances/`, `urban-projects/`, `infrastructure-records/`, `permit-applications/`.

Barangay / GIS Mapping

Built with **Leaflet.js** as the client-side interactive map library — no Google Maps, no paid mapping SDK. The underlying geographic data was sourced from **OpenStreetMap** (188 barangay boundary polygons via a MIT-licensed PSA/PSGC dataset, plus 74,967 building footprints and 348 named subdivisions fetched from OSM's public Overpass API) through one-off import scripts — this is a data-import step, not a live runtime dependency; the running app never calls OpenStreetMap directly. Because 74,967 building rows is too much to hand to Leaflet at once, the map is **zoom-gated and bbox-filtered**: it only loads the Subdivisions/Housing Projects layer above zoom level 13, and only queries Buildings (a debounced bounding-box query on moveend/zoomend) at zoom 15+, with **BuildingService** also hard-capping any single query at 2000 rows and clamping the box to Caloocan's extent. There's no MySQL spatial index — plain indexed DECIMAL lat/lng columns are pre-filtered server-side by bounding box, then Leaflet renders the exact polygon shapes client-side from the returned rows. The map itself is read-only (no create/edit/delete flow).

Built in: `BarangayRepository.php`, `SubdivisionRepository.php`, `BuildingRepository.php`, `HousingProjectRepository.php`; `BarangayService.php`, `SubdivisionService.php`, `BuildingService.php`, `HousingProjectService.php`; `api/employee/barangays.php`, `subdivisions.php`, `buildings.php`, `housing-projects.php`; `pages/urban-planning/mapping.php`, `subdivisions.php`, `housing-projects.php`; `assets/js/mapping/`, `subdivisions/`, `housing-projects/`.

Cross-cutting modules

- **Activity Log** — `ActivityLogService::record()` is called explicitly from the create/update/archive/delete path of all 15 local-DB write endpoints, wrapped in try/catch so a logging failure can never break the write it's attached to. Exposed as a filterable, read-only page. Files: `ActivityLogRepository.php`, `ActivityLogService.php`, `api/employee/activity-logs.php`, `pages/logs/activity.php`, `assets/js/logs/`.
- **Program Analytics** — no new tables; `AnalyticsRepository` runs cross-module SQL aggregation directly (occupancy rate, survey completion rate, urban project completion rate as computed KPI math, plus a UNION ALL query stitching a recent-activity feed across residents/housing/projects/surveys). Charts drawn with **Chart.js**, loaded via a direct script tag rather than the CRUD bridge system since the page has no create/edit/delete flow. Files: `AnalyticsRepository.php`, `api/employee/analytics.php`, `pages/analytics/overview.php`, `assets/js/analytics/`.
- **AI Planning Assistant** — `GeminiService` calls Google's Gemini generateContent/chat REST API directly over cURL (matching the existing proxy's cURL conventions instead of adding an HTTP client library). Every request is grounded by injecting a live `AnalyticsRepository` snapshot into Gemini's systemInstruction, so the assistant answers from the real current numbers instead of inventing them. Conversation history sent to the API is capped at the last 20 turns to bound payload size. Files: `GeminiService.php`, `api/employee/ai-assistant.php`, `pages/ai-assistant/chat.php`, `assets/js/ai-assistant/`.
- **Reports** — no new tables or query logic at all; it calls the same `paginate()/stats()` methods every other repository already exposes. The one deliberate exception is CSV export, which bypasses pagination entirely and streams every matching row with `fputcsv`, trading response size for a complete export. Files: `api/employee/reports.php`, `pages/reports/{resident,housing,project,survey}-reports.php`, `assets/js/reports/` (Chart.js direct-load, same pattern as Analytics).

Citizen-facing backend (local)

A separate, simpler identity system from staff accounts. Passwords are hashed with PHP's `password_hash()/password_verify()` (bcrypt). Because citizens aren't staff sessions, these endpoints bootstrap through `config/citizen_app_bootstrap.php` instead of the full `src/bootstrap.php`. Login has brute-force lockout (5 failed attempts → 15-minute lock). CORS is opened specifically for private LAN IP ranges so the companion Expo mobile app's dev client can reach the API from a phone on the same network during development.

Built in: `CitizenAccountRepository.php`, `CitizenAccountService.php`; `api/citizen-app/*.php` (register, login, logout, profile, beneficiary-documents, set-password, forgot-password, housing-projects, housing-programs); `pages/citizen-app/set-password.php` (web fallback — the primary client is the sibling mobile app).

Production-proxied modules

These aren't "built" in this repo in the usual sense — there's no local table, repository, or service. Each API file just forwards the request to production (`config/proxy.php`'s `proxyRequest()`, over cURL) and relays the session cookie kept in `$_SESSION['remote_phpsessid']`. A few add a local guard before proxying — e.g. `users.php` blocks a non-superadmin from creating an admin account — but the actual data always lives on `civentral.tech`, never here. This pattern covers:

- **Login / OTP** — `login.php`, `verify-otp.php`, `resend-otp.php`.
- **User Management** — `users.php`, pages under `pages/usermanagement/`.
- **Roles & Permissions** — `roles.php`, `modules.php`, `resources.php`, `actions.php`, `permissions.php`, `access-control.php`, pages under `pages/rolespermission/`.
- **Department Management** — `departments.php`, `pages/department/departments.php`.
- **Profile & Notifications** — `profile.php`, `get-profile.php`, `change-password.php`, `notifications.php`, `pages/profile/` — reachable by every logged-in user, not gated behind a permission, same as Dashboard.
- **Audit Logs** — `audit-logs.php`, `login-history.php`, `pages/audit/` — the production-side audit trail, distinct from this project's own local Activity Log above.
- **Citizen Account Directory (production)** — `api/citizen/*.php`, `pages/citizen/` — the production citizen directory, unrelated to this project's own local `citizen_accounts` above.

None of these have a Module Management/Permission Builder step of their own — those modules and resources already exist in production.

Standard scaffolding recipe (for a new local-DB module)

1. `database/x.sql` — new table(s), appended to the apply-order list in `setup.md`.
2. `src/Repositories/XRepository.php` — `find/paginate/stats/create/update/setStatus`.
3. `src/Services/XService.php` — validation + whitelist-sanitize + business rules.
4. Register both in `src/bootstrap.php` (constructor-injected, no autoloader).

5. `api/employee/x.php` — require `src/bootstrap.php`, `PermissionMiddleware::requireResource('...')`, REST verbs, `respond([...])`.
6. `pages/<area>/x.php` — sets `$basePath`, includes `header/sidebar/footer`.
7. `assets/js/<module>/{api,table,modal,filters,events,app}.js` — bridge, registered in `assets/js/app.js`, bump `ASSET_VERSION`.
8. Sidebar entry in `includes/sidebar.php`, gated with `$hasResourceAccess([...keywords])`.
9. Create the matching module + resource in production Module Management, grant it to roles in Permission Builder — this project has no local permission table of its own.